Variational inference

ELG 5218 - Uncertainty Evaluation in Engineering Measurements and Machine Learning

Miodrag Bolic

University of Ottawa

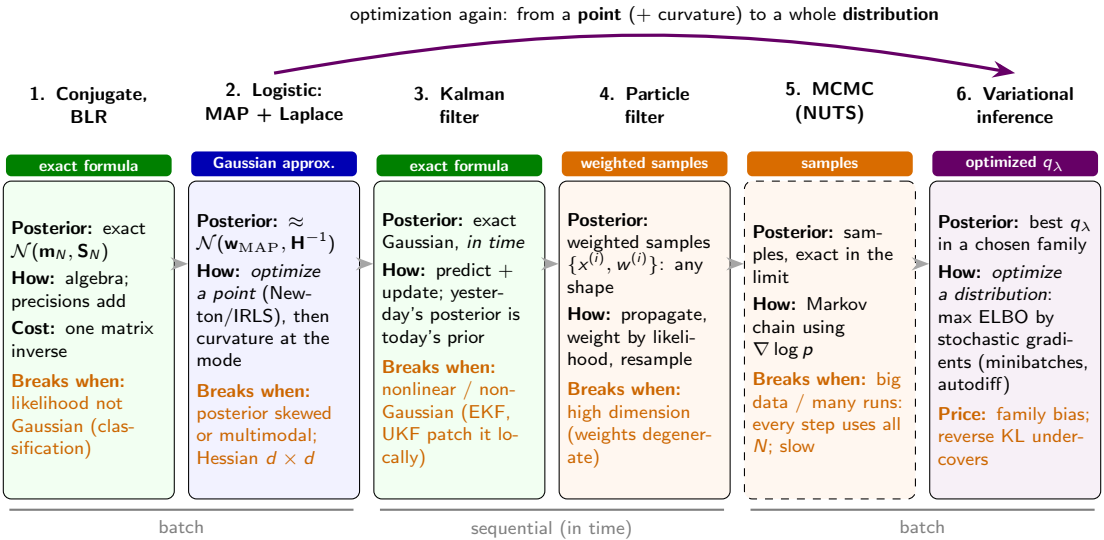
November 2, 2026

Outline

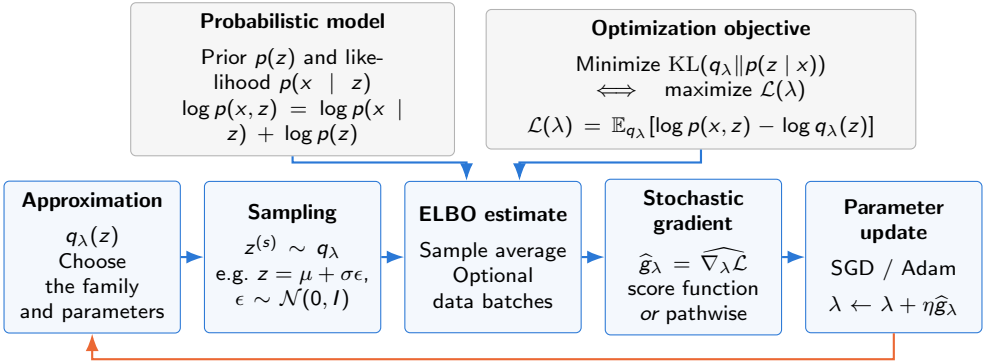
1. Gradients
   - Stochastic Optimization (SGD)
   - Automatic Differentiation (Autodiff)
2. Entropy
3. KL
4. Intro to variational distribution
5. Mean-field family and CAVI
6. Automated VI: stochastic gradients of the ELBO
   - Pathwise gradients of the ELBO
7. Automated VI in practice

Notebook: From_entropy_to_SVI$_2$.*ipynb*

How we have computed posteriors so far



Variational inference: how the concepts connect



$$\log p(x) = \mathcal{L}(\lambda) + \mathrm{KL}(q_\lambda(z) \| p(z \mid x))$$

Fixed model: **ELBO increases $\iff$ posterior KL decreases**.

VI turns posterior approximation into an optimization problem.

Gradient descent and ascent (recap)

The **gradient** $\nabla_\theta f(\theta) = (\partial f/\partial\theta_1, \ldots, \partial f/\partial\theta_d)^\top$ points in the direction of steepest *increase*.

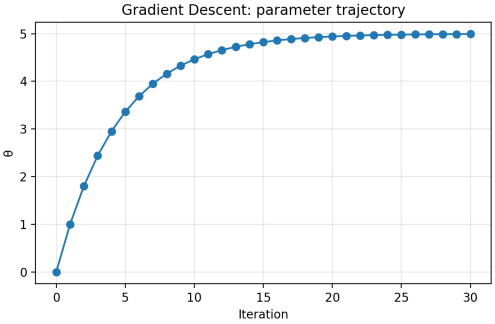
- **Minimize** a loss (descent): $\theta \leftarrow \theta - \eta\, \nabla_\theta f(\theta)$
- **Maximize** an objective (ascent): $\theta \leftarrow \theta + \eta\, \nabla_\theta \mathcal{L}(\theta)$

In VI we **maximize the ELBO** $\mathcal{L}(\lambda)$; software minimizes $-\mathcal{L}(\lambda)$. Same step.

Example. $f(\theta) = (\theta - 5)^2$, $f'(\theta) = 2(\theta - 5)$:

$$\theta_{k+1} = \theta_k - \eta\, 2(\theta_k - 5) \;\longrightarrow\; \theta^\star = 5.$$

Left of 5 the gradient is negative, so the step moves right, and vice versa.



Definition: full gradient vs. stochastic gradient

Let the empirical risk be

$$f(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; x_i, y_i).$$

where $\ell(\theta; x_i, y_i)$ is the **loss** of the model on example i (e.g. squared error $(y_i - \theta^\top x_i)^2$, or log-loss $-\log p_\theta(y_i \mid x_i)$). f is a sample average approximating the *true risk* $R(\theta) = \mathbb{E}_{(x,y)\sim p_{\text{data}}}[\ell(\theta; x, y)]$. Loss ℓ measures how costly one decision or prediction is, and the risk is the expected loss. **Full (batch) gradient:**

$$\nabla f(\theta) = \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \ell(\theta; x_i, y_i).$$

Stochastic gradient (single sample): pick i_t and use

$$g_t = \nabla_\theta \ell(\theta_t; x_{i_t}, y_{i_t}), \qquad \theta_{t+1} = \theta_t - \eta_t \, g_t.$$

Mini-batch stochastic gradient (batch size k)

Pick a mini-batch $B_t \subset \{1, \dots, n\}$ with $|B_t| = k$ and use

$$g_t = \frac{1}{k} \sum_{i \in B_t} \nabla_\theta \ell(\theta_t; x_i, y_i), \qquad \theta_{t+1} = \theta_t - \eta_t \, g_t.$$

- **Variance reduction:** gradient noise typically decreases as $\approx 1/k$.
- **Compute tradeoff:** each step costs k examples, but can be faster in wall-clock time on GPUs/TPUs.
- **Two common sampling modes:**
  - *With replacement*: indices i.i.d. uniform each step.
  - *Without replacement within an epoch*: shuffle once per epoch, then take consecutive slices of the shuffled order (common in ML code).
    - **Example**, $n = 10$, $k = 3$: shuffle $\pi = (7, 2, 9, 4, 1, 10, 3, 6, 8, 5) \Rightarrow B_1 = \{7, 2, 9\}$, $B_2 = \{4, 1, 10\}$, $B_3 = \{3, 6, 8\}$, $B_4 = \{5\}$. After 4 steps, every example has been used once: that's one epoch. Then reshuffle and repeat
    - **Example**, Number of epochs E or total optimizer steps $T = E\lceil n/k \rceil$ (e.g. $n = 50{,}000$, $k = 128$, $E = 10 \Rightarrow T = 3910$).

What parameters do you choose in SGD?

Core knobs

- **Batch size** k (or B): e.g., 1, 32, 128, 1024.
- **Learning rate** η and **schedule**: constant, step decay, cosine, warmup, etc.
- **Number of epochs** E (passes through data) or total steps T.
- **Shuffling / sampling policy**: shuffle each epoch; with/without replacement; sequential windows for time series.
- **Optimizer**: plain SGD, SGD+momentum, or Adam (per-parameter adaptive steps).

Common practical knobs (often essential in deep learning)

- **Momentum** / **Nesterov** (SGD variants).
- **Weight decay** (L2 regularization), dropout (model-level).
- **Gradient clipping** (stability), especially with RNNs/transformers.
- **Early stopping** based on validation performance.

Algorithm: shuffle each epoch + mini-batches (no replacement within epoch)

The standard training loop in ML frameworks (`shuffle=True`). Total number of steps: $T = E\,\lceil n/k \rceil$.

Algorithm 1 Mini-batch SGD with epoch shuffling (typical in ML)

Require: Dataset $\{(x_i, y_i)\}_{i=1}^{n}$, batch size k, epochs E, learning rates $\{\eta_t\}_{t=0}^{T-1}$

Initialize parameters θ; $t \leftarrow 0$
for $e = 1$ to E **do**
  Sample a random permutation π of $\{1, \ldots, n\}$ ▷ shuffle indices
  for $b = 1$ to $\lceil n/k \rceil$ **do**
    $B \leftarrow \{\pi[(b-1)k + 1], \ldots, \pi[\min(bk, n)]\}$ ▷ next slice (last may be shorter)
    $g \leftarrow \frac{1}{|B|} \sum_{i \in B} \nabla_\theta \ell(\theta; x_i, y_i)$
    $\theta \leftarrow \theta - \eta_t\, g$; $t \leftarrow t + 1$
  end for
end for

Every example is used **exactly once per epoch**.

Automatic Differentiation (autodiff) and its relation to SGD

Definition (autodiff). Automatic differentiation is a set of techniques that compute *exact derivatives* (of a function implemented as code) by applying the chain rule to the program's computational graph. Unlike:

- *symbolic differentiation* (manipulates formulas), and
- *numerical differentiation* (finite differences, approximate),

autodiff produces derivatives accurate up to floating-point rounding.

Forward and reverse mode.

- **Forward-mode:** efficient when #inputs is small.
- **Reverse-mode (backpropagation):** efficient when #outputs is small (e.g., scalar loss).

How autodiff connects to batch vs. stochastic gradients. Let the empirical risk be

$$f(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; x_i, y_i).$$

Autodiff is the engine that computes $\nabla_\theta \ell(\theta; x_i, y_i)$ *for whatever data you feed in*.

- **Full-batch GD:** feed all n points, autodiff returns $\nabla f(\theta)$.
- **Mini-batch SGD:** feed a batch B of size k, autodiff returns

$$\nabla_\theta \Big(\frac{1}{k} \sum_{i \in B} \ell(\theta; x_i, y_i) \Big) = \frac{1}{k} \sum_{i \in B} \nabla_\theta \ell(\theta; x_i, y_i).$$

Key takeaway: *Autodiff computes gradients; batching controls* **which** *data points contribute to the gradient estimate.*

Reverse-mode autodiff: forward values, then adjoints backward

Example: $f(\theta) = (\theta - 5)^2$ at the current value $\theta = 7$. The program is split into elementary operations: $u = \theta - 5$, $f = u^2$.

Adjoint (sensitivity) of a node v: $\bar{v} := \partial f / \partial v$, i.e. how much the output f changes per unit change of v.

	Step	Computation	Value
	θ	given	7
Fwd $\downarrow$	$u = \theta - 5$	$7 - 5$	2 (stored)
	$f = u^2$	2^2	4 (stored)
	$\bar{f}$	seed: $\partial f / \partial f$	1
Rev $\uparrow$	$\bar{u}$	$\bar{f} \cdot \frac{\partial f}{\partial u} = 1 \cdot 2u$	4
	$\bar{\theta}$	$\bar{u} \cdot \frac{\partial u}{\partial \theta} = 4 \cdot 1$	**4**

Result: $\left.\frac{df}{d\theta}\right|_{\theta=7} = \bar{\theta} = 4$ (check: $2(7-5) = 4$).

PyTorch: `f = (theta-5)**2; f.backward()`
$\Rightarrow$ `theta.grad` $= 4$.

- Autodiff returns a **number at the current** θ, not the formula $2(\theta - 5)$. The next SGD step runs it again at the new θ.
- Each reverse step is one chain-rule step: $\overline{\text{input}}$ += $\overline{\text{output}} \times$ local derivative. "+ =" sums contributions when a node feeds several operations.
- $\bar{f} = 1$ is a *rate*, not the value $f = 4$.
- Forward values enter only through local derivatives ($2u$ needs $u = 2$), so they must be **stored**: this is why training uses a lot of memory.

Shannon Entropy: Definition

Discrete random variable

Let X be a discrete random variable taking values $x_1, \ldots, x_n$ with probabilities $p_i = \mathbb{P}(X = x_i)$ and $\sum_{i=1}^{n} p_i = 1$.

The **Shannon entropy** of X (in base b) is

$$H_b(X) = -\sum_{i=1}^{n} p_i \log_b p_i.$$

- If we use $\log_2$, the units are **bits**.
- If X is deterministic (one outcome has probability 1), then $H(X) = 0$.
- Entropy measures the **average uncertainty** about X.

Surprise: the building block of entropy

The **surprise** (self-information) of an outcome x:

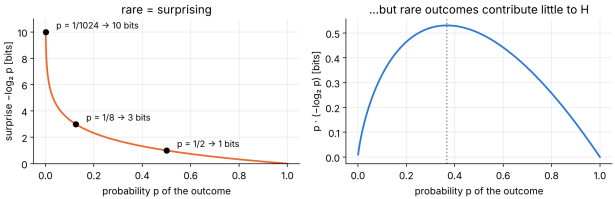
$$s(x) = -\log_2 p(x) \quad \text{[bits]}.$$

- $p = 1$: $s = 0$, a certain event tells us nothing.
- $p = \frac{1}{2}$: 1 bit; $p = \frac{1}{8}$: 3 bits; $p = \frac{1}{1024}$: 10 bits.
- 1 bit = surprise of one fair coin flip; s bits = as surprising as s heads in a row.

Entropy = expected surprise

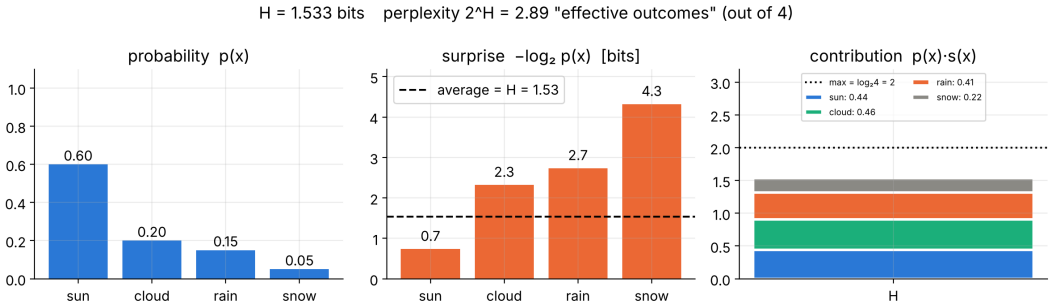
$$H(X) = \mathbb{E}\big[s(X)\big] = -\sum_x p(x)\log_2 p(x).$$

Rare outcomes are very surprising but rarely happen: $p \log p \to 0$ as $p \to 0$.



ML link: the log-loss of a classifier is the surprise $-\log q(y)$ of the true label under the model; Langevin dynamics, HMC and diffusion models all move along $-\nabla_x s(x) = \nabla_x \log p(x)$ (the score)

Example: a weather forecast, average surprise and perplexity



- $p = (0.6, 0.2, 0.15, 0.05)$: $H = 1.53$ bits (maximum for 4 outcomes: $\log_2 4 = 2$ bits).
- **Perplexity** $2^H = 2.89$: the forecast is as uncertain as a fair die with ≈ 2.9 faces (*effective number of outcomes*). Perplexity says "you're as uncertain as if choosing among 2.89 equally likely options. Language models report exactly this number.

▶ **Notebook demo A1** sliders: weight of each outcome. Try making "snow" very rare: its surprise explodes, its contribution to H shrinks.

Differential Entropy and the Normal Distribution

For a continuous random variable X with density $p(x)$, the **differential entropy** (in nats meaning that the $log = ln$) is

$$h(X) = -\int_{-\infty}^{\infty} p(x) \log p(x)\, dx.$$

Entropy of a normal distribution

If $X \sim \mathcal{N}(\mu, \sigma^2)$, then

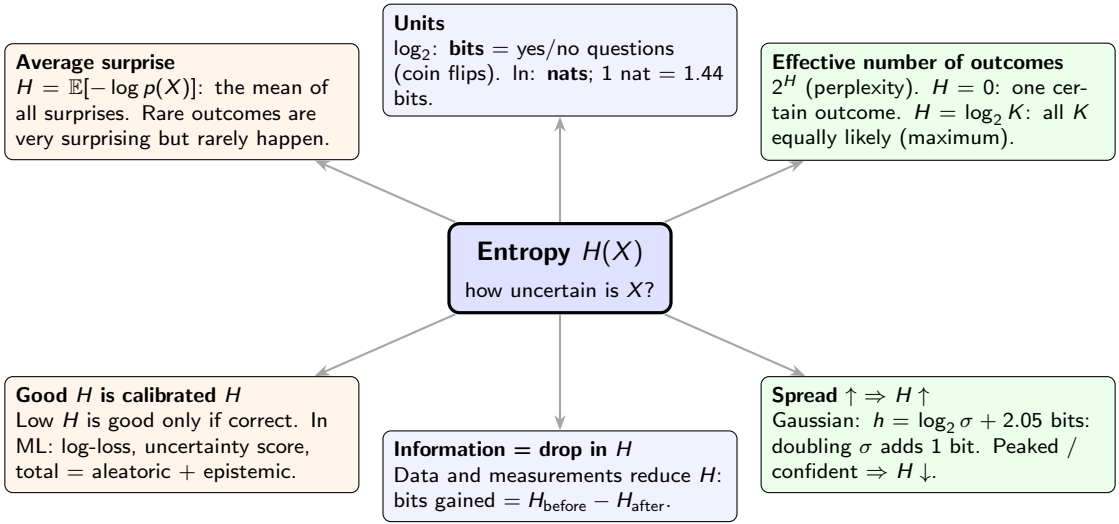
$$h(X) = \frac{1}{2} \log\left(2\pi e \sigma^2\right).$$

Maximum entropy principle (continuous case)

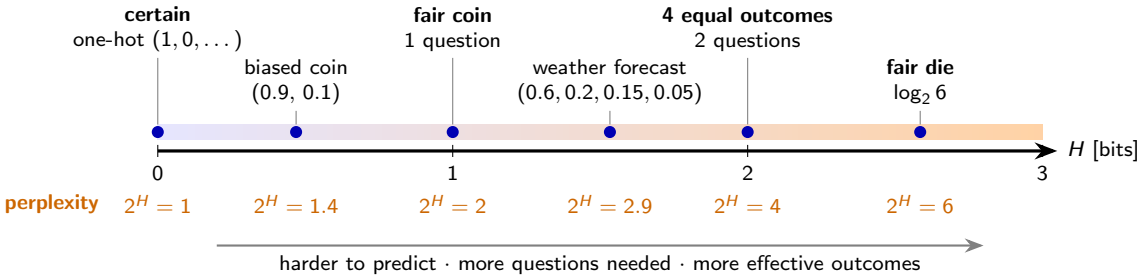
Among all real-valued random variables with a given mean μ and variance σ^2, the normal distribution $\mathcal{N}(\mu, \sigma^2)$ has the **largest** differential entropy.

- The normal distribution is the "most spread out" shape compatible with the specified mean and variance.
- In that sense, it is the **most uncertain** distribution under these constraints.

Entropy in one picture



What does an entropy value mean? A ruler in bits



- $H = 0$: the outcome is known in advance; observing it teaches nothing. $H = 1$ **bit**: as uncertain as one fair coin. $H = \log_2 K$: maximum, all K outcomes equally likely.
- 4 equal outcomes: one roll of a fair die with 4 sides resolves as much uncertainty as 2 fair coin flips and therefore $H = 2$.
- **Continuous variables**: $h = \log_2 \sigma + 2.05$ bits for a Gaussian, so each doubling of σ adds 1 bit. h can be negative; only *differences* (information gained) have a physical meaning.

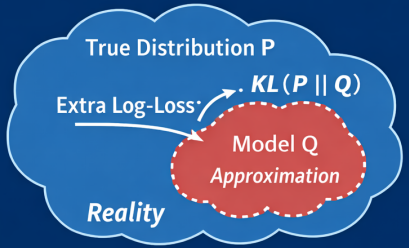
Kullback–Leibler divergence: definition and properties

Definition

$$\mathrm{KL}(P\|Q) = \sum_i p_i \log \frac{p_i}{q_i} \qquad \text{(discrete)}, \qquad \mathrm{KL}(P\|Q) = \int p(x) \log \frac{p(x)}{q(x)}\, dx \qquad \text{(continuous)}.$$

- Measures how different Q (model) is from P (truth), averaged *under* P. Units: bits ($\log_2$) or nats (ln).
- **Non-negative:** $\mathrm{KL}(P\|Q) \geq 0$, with equality iff $P = Q$.
- **Not symmetric:** in general $\mathrm{KL}(P\|Q) \neq \mathrm{KL}(Q\|P)$ (this matters for VI, see below).
- **Support sensitive:** if $q = 0$ where $p > 0$, then $\mathrm{KL}(P\|Q) = \infty$.
- Closed form between Gaussians; otherwise estimated by Monte Carlo (sample from the distribution in front).

Intuition



- P = True distribution
- Q = Model / Approximation
- KL(P || Q) = Expected *extra log-loss* using Q instead of P
- Large KL ➡ *Very wasteful encoding with Q*
- Small KL ➡ *Q close to P*

Asymmetry

$$KL(P \,||\, Q) \neq KL(Q \,||\, P).$$

Changing the roles of "true" and "model" matters.

Entropy, Cross-Entropy, and KL as *Expected Log-Loss*

Setup. Reality generates outcomes i from the true distribution P (probabilities p_i). We predict using a model Q (probabilities q_i). Consider the **log-loss** (negative log-likelihood) for outcome i: $\ell_Q(i) = -\log q_i$.

Interpretations

- **Entropy** $H(P) = -\sum_i p_i \log p_i = \mathbb{E}_{i\sim P}[-\log p_i]$: the *irreducible average surprise* in outcomes drawn from P.
- **Cross-entropy** $H(P, Q) = -\sum_i p_i \log q_i = \mathbb{E}_{i\sim P}[-\log q_i]$: the *average log-loss you incur* when using model Q on data from P.

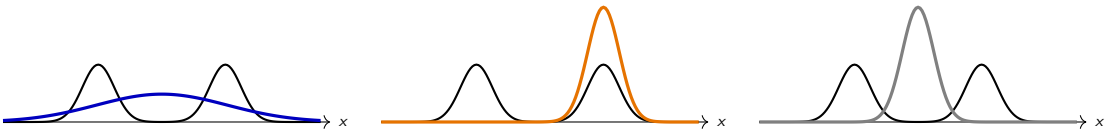
Decomposition (mismatch penalty)

$$\mathrm{KL}(P\|Q) = \sum_i p_i \log \frac{p_i}{q_i} = H(P, Q) - H(P).$$

- $\mathrm{KL}(P\|Q)$: **extra expected log-loss** due to using the wrong probabilities Q.
- Minimizing cross-entropy $H(P, Q)$ is equivalent to minimizing $\mathrm{KL}(P\|Q)$ (since $H(P)$ is fixed).

Why order matters (1/2): which single Gaussian is "closest" to p?

Task. Target with two modes: $p(x) = 0.5\,\mathcal{N}(-2, 0.5^2) + 0.5\,\mathcal{N}(2, 0.5^2)$. We may only use **one** Gaussian $q(x) = \mathcal{N}(\mu, \sigma^2)$. "Best approximation" = the (μ, σ) with the **smallest KL**. No algorithm needed yet: just compare candidates.



	A wide	**B** one bump	**C** valley
forward $\mathrm{KL}(p\|q)$ [nats]	**0.72** (best)	15.3	7.3
reverse $\mathrm{KL}(q\|p)$ [nats]	2.10	**0.69** (best)	5.4

Black: target p. **The two directions pick different winners.** Forward KL picks A, which **covers both modes** (puts probability everywhere p does). Reverse KL picks B, which **chooses one mode** (and ignores the other). A is in fact the best Gaussian for forward KL (μ and σ match the mean and std of p).

Why order matters (2/2): what each direction punishes

Both average a log-ratio, but **weighted by different distributions** (coloured):

$$\mathrm{KL}(p\|q) = \int p(x) \log \frac{p(x)}{q(x)}\, dx \qquad \text{vs} \qquad \mathrm{KL}(q\|p) = \int q(x) \log \frac{q(x)}{p(x)}\, dx$$

Forward $\mathrm{KL}(p\|q)$: "miss nothing"

- Only places where p has mass count. There, $q \approx 0$ gives $\log \frac{p}{q} \to \infty$: **missing a bump is fatal** (B: 15.3).
- In the valley $p \approx 0$: q may waste mass there for free.

$\Rightarrow$ one wide q over both bumps: **mass-covering**.

Reverse $\mathrm{KL}(q\|p)$: "claim nothing false"

- Only places where q has mass count. There, $p \approx 0$ gives $\log \frac{q}{p} \to \infty$: **mass in the valley is fatal** (C: 5.4).
- Ignoring the other bump ($q \approx 0$ there) costs nothing.

$\Rightarrow$ one Gaussian cannot cross the valley: it sits on one bump, **mode-seeking**.

- **Analogy**, hiker lost in one of two valleys: forward searches both valleys and the ridge; reverse, one valley.
- B's reverse KL is $0.69 = \ln 2$ nats $= 1$ bit: the price of not knowing *which* bump.
- **Why it matters:** VI uses the reverse KL $\Rightarrow$ may drop modes, **underestimate uncertainty**.

Why VI uses reverse KL $\mathrm{KL}(q\|p(z \mid x))$

Variational inference chooses $q_\lambda(z)$ and minimizes

$$\mathrm{KL}(q_\lambda(z)\|p(z \mid x)).$$

This direction is convenient because the expectation is under q:

- we can sample from q and evaluate $\log p$ up to a constant,
- this yields the ELBO objective.

Consequence: mode-seeking behavior can cause *mode dropping* for multi-modal posteriors.

Setup and Notation for Variational Inference

We consider a probabilistic model with:

- Latent variable z
- Observed data x
- Joint distribution $p(x, z)$
- Posterior distribution $p(z \mid x)$

Key quantities

- **Posterior:**

$$p(z \mid x) = \frac{p(x, z)}{p(x)}.$$

- **Evidence (marginal likelihood):**

$$p(x) = \int p(x, z)\, dz.$$

In many interesting models, $p(z \mid x)$ is intractable. **Variational inference (VI)** approximates $p(z \mid x)$ with a simpler distribution $q(z)$ and finds the best q by optimization.

Introducing a Variational Distribution

We want to approximate the posterior $p(z \mid x)$.

- Choose a variational family $\mathcal{Q} = \{q(z; \lambda)\}$, e.g. Gaussians with parameters λ.
- Goal: find $q^*(z) \in \mathcal{Q}$ that is as close as possible to the true posterior by finding the optimal value of λ: $q^*(z) = \arg\min_\lambda \mathrm{KL}(q_\lambda(z) \,\|\, p(z \mid x))$

Start from the marginal likelihood:

$$\log p(x) = \log \int p(x, z)\, dz.$$

Multiply and divide by any density $q(z)$:

$$\log p(x) = \log \int q(z) \frac{p(x, z)}{q(z)}\, dz = \log \mathbb{E}_{q(z)}\left[\frac{p(x, z)}{q(z)}\right].$$

Next: apply Jensen's inequality.

Deriving the ELBO via Jensen's Inequality

Recall Jensen's inequality for a concave function f:

$$f\big(\mathbb{E}[Y]\big) \;\geq\; \mathbb{E}[f(Y)].$$

For $f = \log$ (concave) and

$$Y = \frac{p(x,z)}{q(z)},$$

we obtain

$$\log \mathbb{E}_{q(z)}\left[\frac{p(x,z)}{q(z)}\right] \;\geq\; \mathbb{E}_{q(z)}\left[\log \frac{p(x,z)}{q(z)}\right].$$

Therefore

$$\begin{aligned}\log p(x) &= \log \mathbb{E}_{q(z)}\left[\frac{p(x,z)}{q(z)}\right] \\ &\geq \mathbb{E}_{q(z)}[\log p(x,z) - \log q(z)]\,.\end{aligned}$$

Evidence Lower Bound (ELBO)

$$\mathcal{L}(q) := \mathbb{E}_{q(z)}[\log p(x,z)] - \mathbb{E}_{q(z)}[\log q(z)],$$

so that

$$\log p(x) \;\geq\; \mathcal{L}(q).$$

Gap to evidence: $\log p(x) = \mathcal{L}(q) + \mathrm{KL}(q(z) \| p(z \mid x))$

Derivation

Start from the KL divergence to the true posterior:

$$\mathrm{KL}(q(z) \,\|\, p(z \mid x)) = \mathbb{E}_q\left[\log \frac{q(z)}{p(z \mid x)}\right] = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(z \mid x)].$$

Use Bayes' rule $p(z \mid x) = \frac{p(x,z)}{p(x)}$, so $\log p(z \mid x) = \log p(x, z) - \log p(x)$. Plug in:

$$\mathrm{KL}(q \| p(\cdot \mid x)) = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(x, z) - \log p(x)] = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(x, z)] + \log p(x).$$

Rearrange:

$$\log p(x) = \underbrace{\mathbb{E}_q[\log p(x, z)] - \mathbb{E}_q[\log q(z)]}_{\mathcal{L}(q)} + \mathrm{KL}(q(z) \,\|\, p(z \mid x)) \,.$$

Key takeaway

Since $\mathrm{KL}(\cdot) \geq 0$, we have $\log p(x) \geq \mathcal{L}(q)$, and maximizing the ELBO is equivalent to minimizing $\mathrm{KL}(q(z) \| p(z \mid x))$.

Alternative Form of the ELBO

Decompose the joint:

$$\log p(x, z) = \log p(x \mid z) + \log p(z).$$

Plug into the ELBO:

$$\begin{aligned}\mathcal{L}(q) &= \mathbb{E}_{q(z)}[\log p(x, z)] - \mathbb{E}_{q(z)}[\log q(z)] \\ &= \mathbb{E}_{q(z)}[\log p(x \mid z)] + \mathbb{E}_{q(z)}[\log p(z)] - \mathbb{E}_{q(z)}[\log q(z)] \\ &= \mathbb{E}_{q(z)}[\log p(x \mid z)] - \mathrm{KL}(q(z) \| p(z)).\end{aligned}$$

Interpretation

- Maximizing $\mathcal{L}(q)$ gives a q that explains data well ($\mathbb{E}_{q(z)}[\log p(x \mid z)]$) and is close to the prior ($\mathrm{KL}(q(z) \| p(z))$).
- $\mathbb{E}_{q(z)}[\log p(x \mid z)]$ is the expected log-likelihood under the variational posterior.
- $\mathrm{KL}(q(z) \| p(z))$ penalizes deviation from the prior.
- VI trades off data fit vs. regularization toward the prior.

The mean-field family, and its classic optimizer (CAVI)

Mean-field is a choice of family: the latent components are independent under q,

$$q(z) = \prod_{i=1}^{m} q_i(z_i).$$

It says *what shape* q may have, not *how* we optimize it.

- $+$ Cheap and scalable: expectations factorize; few parameters.
- $-$ Ignores posterior correlations $\Rightarrow$ typically **underestimates** uncertainty.

Coordinate ascent VI (CAVI). Setting the functional derivative of the ELBO w.r.t. q_i to zero gives the best factor when the others are fixed:

$$\boxed{\log q_i^*(z_i) = \mathbb{E}_{q_{-i}}[\log p(x, z)] + \text{const}}$$

- q_i^* mimics the conditional $p(z_i \mid x, z_{-i})$, with z_{-i} averaged under q_{-i}, where $q_{-j} = \prod_{i \neq j} q_i$.
- Cycle $i = 1, \ldots, m$ until the ELBO stops increasing; each update cannot decrease it.
- Needs $\mathbb{E}_{q_{-i}}[\log p(x, z)]$ in **closed form** (conditionally conjugate models). The form of each q_i (Gaussian, Gamma, ...) is then *derived*, not chosen.

Mean-Field VI: A Simple Gaussian Example

Model:

$$x_n \sim \mathcal{N}(\mu, \tau^{-1}), \qquad p(\mu \mid \tau) = \mathcal{N}(\mu_0, (\lambda_0 \tau)^{-1}), \qquad p(\tau) = \mathrm{Gamma}(a_0, b_0).$$

Goal: Illustrate coordinate ascent VI (even though the true posterior is conjugate).

Mean-field assumption:

$$q(\mu, \tau) = q_\mu(\mu)\, q_\tau(\tau).$$

Coordinate ascent updates:

$$\boxed{\log q_\mu^*(\mu) = \mathbb{E}_{q_\tau}[\log p(X, \mu, \tau)] + \text{const}}$$

$$\boxed{\log q_\tau^*(\tau) = \mathbb{E}_{q_\mu}[\log p(X, \mu, \tau)] + \text{const}}$$

Idea: Each factor matches the model's conditional distribution, averaged over the other variational factors.

Updating $q_\mu(\mu)$

Log-joint terms involving μ:

$$\log p(X, \mu, \tau) = -\frac{\tau}{2}\sum_{n=1}^{N}(x_n - \mu)^2 - \frac{\lambda_0\tau}{2}(\mu - \mu_0)^2 + C.$$

Apply MF update:

$$\log q^*_\mu(\mu) = -\frac{\mathbb{E}_{q_\tau}[\tau]}{2}\left[\sum_{n=1}^{N}(x_n - \mu)^2 + \lambda_0(\mu - \mu_0)^2\right] + \text{const.}$$

Recognize a Gaussian:

$$q^*_\mu(\mu) = \mathcal{N}(\mu_N, \tau_N^{-1}),$$

$$\mu_N = \frac{\lambda_0\mu_0 + N\bar{x}}{\lambda_0 + N}, \qquad \tau_N = (\lambda_0 + N)\,\mathbb{E}_{q_\tau}[\tau].$$

Note: q_μ depends on $\mathbb{E}[\tau]$, so we alternate updates.

Updating $q_\tau(\tau)$

Log-joint terms involving τ:

$$\log p(X, \mu, \tau) = \left(a_0 - 1 + \frac{N+1}{2}\right) \log \tau - \tau \left[b_0 + \frac{1}{2}\Big(\sum_{n=1}^{N}(x_n - \mu)^2 + \lambda_0(\mu - \mu_0)^2\Big)\right] + C.$$

Apply MF update:

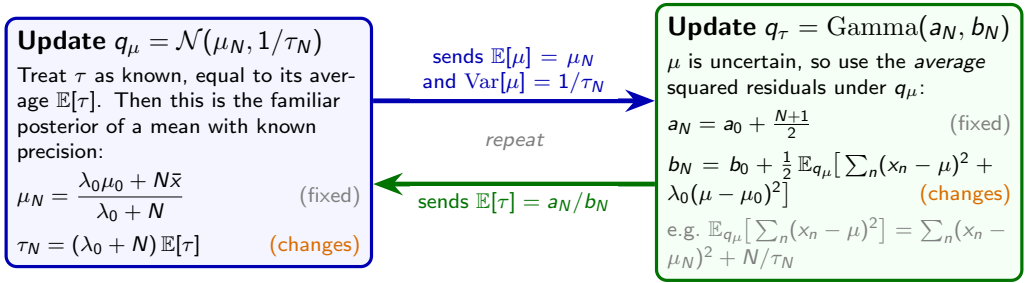
$$q_\tau^*(\tau) = \mathrm{Gamma}(a_N, b_N),$$

$$a_N = a_0 + \frac{N+1}{2}, \qquad b_N = b_0 + \frac{1}{2}\,\mathbb{E}_{q_\mu}\left[\sum_{n=1}^{N}(x_n - \mu)^2 + \lambda_0(\mu - \mu_0)^2\right].$$

Summary:

- q_μ and q_τ have closed-form Gaussian and Gamma updates.
- Updates depend on each other $\Rightarrow$ run *cyclic* coordinate ascent.
- This example shows the classic MFVI pattern: exponentiate expected log-joint and normalize.

CAVI: the two factors pass averages back and forth



The rule behind every CAVI update

Update one factor = the exact posterior of that variable *as if the others were known*, with the others replaced by their **averages** under their current q. Repeat until nothing changes.

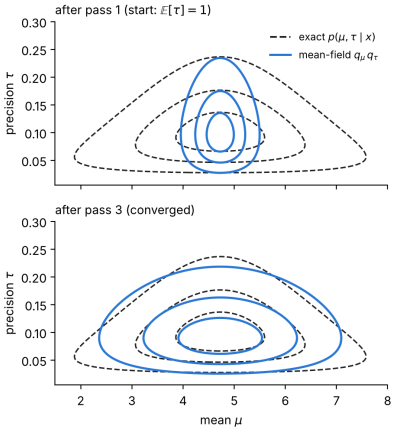
Compact form of the same rule (not needed to compute here): $\log q_j^*(z_j) = \mathbb{E}_{q_{-j}}[\log p(x, z)] + \text{const}$.

CAVI on data: three passes are enough

$N = 10$ readings, $\bar{x} = 4.78$; prior $\mu_0 = 0$, $\lambda_0 = 0.1$, $a_0 = b_0 = 1$; fixed $\mu_N = 4.73$, $a_N = 6.5$; start $\mathbb{E}[\tau] = 1$.

Pass	$\mathbb{E}[\tau]$ used (from q_τ)	sd of q_μ $1/\sqrt{\tau_N}$	new $\mathbb{E}[\tau]$ $= a_N/b_N$
1	1.000	0.315	0.114
2	0.114	0.933	0.107
3	0.107	0.964	0.106
4	0.106	0.967	0.106
exact	0.106	1.059	

- One number travels around the loop: $\mathbb{E}[\tau]$ (and $\mathrm{Var}[\mu]$ back).
- The ELBO never decreases; a bad start is fixed in a pass.
- Means are right; the spread of μ is **9% too narrow** (0.97 vs. 1.06).
- Why: the exact posterior (dashed) is wider in μ when τ is small. $q_\mu\, q_\tau$ cannot represent this *dependence*.



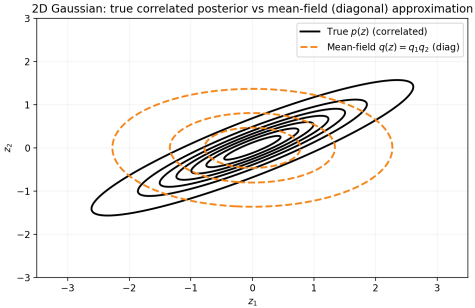
Correlated posterior vs mean-field: geometry of uncertainty (contours)

Setup. True posterior: $p(z) = \mathcal{N}(0, \Sigma)$ with correlation. Mean-field Gaussian: $q(z) = \mathcal{N}(0, \Sigma_q)$ with Σ_q diagonal (equivalently $q(z) = q_1(z_1)q_2(z_2)$).

What we expect. Correlation rotates the principal axes of uncertainty:

- correlated Gaussian $\Rightarrow$ **tilted ellipses** (off-axis principal components),
- diagonal covariance $\Rightarrow$ **axis-aligned ellipses** (no tilt).

- **Observed:** black contours (true p) are tilted.
- **Observed:** dashed contours (mean-field q) are axis-aligned.
- **Meaning:** q cannot "rotate" to match p because independence forbids correlation.



Takeaway. Mean-field captures some spread but misses joint structure $\Rightarrow$ biased uncertainty and wrong dependence. This holds for *any* mean-field q, whichever optimizer (CAVI, score function, pathwise) fitted it.

Two separate decisions: the family q and the way we optimize it

Which family q?	**How do we maximize the ELBO?** **CAVI** closed-form updates	**Score-function gradient** (black-box VI)	**Pathwise gradient** (reparameterization)
Mean-field $\prod_j q_j(z_j)$	classic mean-field VI (conjugate models; this lecture's μ, τ example)	BBVI (Ranganath et al., 2014)	ADVI (Automatic Differentiation) mean-field, Bayes by Backprop; NumPyro `AutoNormal`
Full-covariance Gaussian $\mathcal{N}(\mu, LL^\top)$	not used (CAVI needs factorized q)	possible, but high variance	ADVI full-rank; NumPyro `AutoMultivariateNormal`
Richer mixtures, flows	not used	possible	normalizing flows, VAEs

- **Rows** (mean-field, full-covariance, flow): *what shape* the approximation may have.
- **Columns**: *how* we get $\nabla_\lambda \mathcal{L}$. Score-function and pathwise are **gradient estimators**.
- CAVI is automatic only for conditionally conjugate models (variational message passing, e.g. Infer.NET); otherwise its updates must be derived by hand. The gradient columns are automatic for any model whose log p(x,z) we can evaluate.

Fixed-form VI: choose the shape of q_λ, learn λ by gradient ascent

- CAVI needs closed-form expectations. For most models (logistic regression, neural networks, non-conjugate priors) they are not available.
- **Fixed-form VI:** choose a parametric family $q_\lambda(z)$ and maximize the ELBO over λ:

$$\lambda \leftarrow \lambda + \eta\, \widehat{\nabla_\lambda \mathcal{L}}(\lambda), \qquad \mathcal{L}(\lambda) = \mathbb{E}_{q_\lambda(z)}\big[\log p(x,z) - \log q_\lambda(z)\big].$$

Examples of families $q_\lambda(z)$ (any of them can be used below)

- **Mean-field Gaussian:** $\mathcal{N}(z \mid \mu, \mathrm{diag}(\sigma^2))$, $\lambda = (\mu, \log\sigma)$: $2d$ numbers.
- **Full Gaussian:** $\mathcal{N}(z \mid \mu, \Sigma)$, $\lambda = (\mu, L)$ with $\Sigma = LL^\top$ (Cholesky).
- **Low-rank + diagonal:** $\mathcal{N}(z \mid \mu, aa^\top + D)$, $\lambda = (\mu, a, d)$, $D = \mathrm{Diag}(d)$.
- **Richer:** mixtures of Gaussians, normalizing flows.

The difficulty: in $\nabla_\lambda \mathbb{E}_{q_\lambda}[\cdot]$ the distribution we average over depends on λ. Two ways to estimate this gradient: **score function** (next) and **pathwise**.

Approach 1: the ELBO gradient as an expectation (score-function form)

Define

$$h_\lambda(z) := \log p(x,z) - \log q_\lambda(z) \quad \Longrightarrow \quad \mathcal{L}(\lambda) = \int q_\lambda(z)\, h_\lambda(z)\, dz.$$

Derivation sketch (log-derivative trick)

$$\begin{aligned}\nabla_\lambda \mathcal{L}(\lambda) &= \nabla_\lambda \int q_\lambda(z)\, h_\lambda(z)\, dz = \int \nabla_\lambda (q_\lambda(z)\, h_\lambda(z))\, dz \\ &= \int \Big((\nabla_\lambda q_\lambda(z))\, h_\lambda(z) + q_\lambda(z)\, \nabla_\lambda h_\lambda(z) \Big)\, dz.\end{aligned}$$

Since $\nabla_\lambda h_\lambda(z) = -\nabla_\lambda \log q_\lambda(z)$ and $\nabla_\lambda q_\lambda(z) = q_\lambda(z) \nabla_\lambda \log q_\lambda(z)$, the second term cancels because

$$\int \nabla_\lambda q_\lambda(z)\, dz = \nabla_\lambda \int q_\lambda(z)\, dz = \nabla_\lambda 1 = 0.$$

Therefore,

$$\nabla_\lambda \mathcal{L}(\lambda) = \int q_\lambda(z)\, h_\lambda(z)\, \nabla_\lambda \log q_\lambda(z)\, dz = \mathbb{E}_{q_\lambda(z)}\big[h_\lambda(z)\, \nabla_\lambda \log q_\lambda(z)\big].$$

Score-function (black-box) VI: algorithm and requirements

Monte Carlo estimator

Draw $z^{(1)}, \ldots, z^{(S)} \sim q_\lambda(z)$ and estimate

$$\widehat{\nabla_\lambda \mathcal{L}}(\lambda) = \frac{1}{S} \sum_{s=1}^{S} \nabla_\lambda \log q_\lambda(z^{(s)}) \left(\log p(x, z^{(s)}) - \log q_\lambda(z^{(s)}) \right).$$

Algorithm (stochastic gradient ascent)

1. Initialize λ; choose learning rate η and S.
2. Repeat until the ELBO stabilizes:
   1. sample $z^{(1)}, \ldots, z^{(S)} \sim q_\lambda$;
   2. compute $\widehat{\nabla_\lambda \mathcal{L}}(\lambda)$ as above;
   3. update $\lambda \leftarrow \lambda + \eta \, \widehat{\nabla_\lambda \mathcal{L}}(\lambda)$.

What we need:

- sample from q_λ;
- evaluate $\log q_\lambda(z)$ and $\nabla_\lambda \log q_\lambda(z)$;
- evaluate $\log p(x, z)$ (no derivative of the model).

Black-box: no model-specific derivation. Works for **discrete** z and non-differentiable models. **Price:** high variance $\Rightarrow$ control variates (appendix), larger S.

Approach 2 (pathwise): why sampling breaks differentiation

Goal: show that $\nabla_\lambda \mathbb{E}_{z\sim q_\lambda}[f(z)]$ cannot be computed by naively differentiating through a Monte Carlo sample.

Example: $q_\lambda(z) = \mathcal{N}(z; \lambda, 1)$

Take $\mathcal{L}(\lambda) = \mathbb{E}_{z\sim\mathcal{N}(\lambda,1)}[f(z)]$ and a sample $z^{(s)} \sim \mathcal{N}(\lambda, 1)$.

A sampled *value* does not depend on λ: $\partial z^{(s)}/\partial\lambda = 0$, so naively

$$\frac{\partial}{\partial\lambda} f(z^{(s)}) = f'(z^{(s)}) \cdot 0 = 0.$$

But for $f(z) = z$: $\mathbb{E}[z] = \lambda \Rightarrow \frac{d}{d\lambda}\mathbb{E}[z] = 1$.

Naive MC gradient: 0. True gradient: 1.

Conclusion: sampling hides the dependence on λ; gradients cannot flow through $z \sim q_\lambda$.

Fix (reparameterization):

$$z = \lambda + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, 1).$$

Now z is a differentiable function of λ ($\partial z/\partial\lambda = 1$) and the randomness sits in ϵ, which does not depend on λ.

(Approach 1 avoids the problem differently: it differentiates the density q_λ, not the sample.)

Pathwise (Reparameterization) Estimator: Gradient Formula

Rewrite the expectation using the reparameterization:

$$\mathcal{L}(\lambda) = \mathbb{E}_{z\sim q_\lambda}[f(z)] = \mathbb{E}_{\epsilon\sim p(\epsilon)}\big[f\big(g(\lambda,\epsilon)\big)\big].$$

Differentiate under the expectation (valid by dominated convergence / smoothness):

$$\nabla_\lambda \mathcal{L}(\lambda) = \mathbb{E}_{\epsilon\sim p(\epsilon)}\Big[\nabla_\lambda f\big(g(\lambda,\epsilon)\big)\Big] = \mathbb{E}_\epsilon\Big[\nabla_z f(z)\big|_{z=g(\lambda,\epsilon)}\ \nabla_\lambda g(\lambda,\epsilon)\Big].$$

In VI (ELBO): If $f(z) = \log p(x,z) - \log q_\lambda(z)$, then

$$\nabla_\lambda \mathbb{E}_{q_\lambda}[\log p(x,z) - \log q_\lambda(z)] = \mathbb{E}_\epsilon\Big[\big(\nabla_z \log p(x,z) - \nabla_z \log q_\lambda(z)\big)\,\nabla_\lambda g(\lambda,\epsilon)\Big].$$

This is the reparameterization gradient / pathwise estimator.

Note. $\log q_\lambda(z)$ also depends on λ directly. That extra term, $-\mathbb{E}_{q_\lambda}[\nabla_\lambda \log q_\lambda(z)]$, has mean zero (score identity), so the formula above is unbiased. Autodiff of the code keeps it; both are valid.

From Formula to Algorithm

We want to maximize the ELBO:

$$\mathcal{L}(\lambda) \;=\; \mathbb{E}_{z\sim q_\lambda}\big[\log p(x,z) - \log q_\lambda(z)\big].$$

Stochastic gradient ascent step:

1. Sample noise $\epsilon^{(s)} \sim p(\epsilon)$ for $s = 1, \ldots, S$.
2. Reparameterize $z^{(s)} = g(\lambda, \epsilon^{(s)})$.
3. Compute

$$g_\lambda \;\approx\; \frac{1}{S}\sum_{s=1}^{S}\Big(\nabla_z \log p(x, z^{(s)}) - \nabla_z \log q_\lambda(z^{(s)})\Big)\;\nabla_\lambda g(\lambda, \epsilon^{(s)}).$$

4. Update $\lambda \leftarrow \lambda + \eta\, g_\lambda$.

Gaussian case: $\lambda = (\mu, \sigma)$ and $g(\lambda, \epsilon) = \mu + \sigma\epsilon$, so

$$z = \mu + \sigma\epsilon, \quad \nabla_\mu g = 1, \quad \nabla_\sigma g = \epsilon.$$

Thus

$$\nabla_\mu \mathcal{L} \approx \frac{1}{S}\sum_s \Big(\nabla_z \log p - \nabla_z \log q_\lambda\Big)\Big|_{z^{(s)}}, \qquad \nabla_\sigma \mathcal{L} \approx \frac{1}{S}\sum_s \Big(\nabla_z \log p - \nabla_z \log q_\lambda\Big)\Big|_{z^{(s)}}\epsilon^{(s)}.$$

Worked Example: 1D Gaussian VI

Model:

$$p(z) = \mathcal{N}(0,1), \quad p(x \mid z) = \mathcal{N}(x; z, 1), \quad \text{observed } x.$$

Variational family: $q_\lambda(z) = \mathcal{N}(z; \mu, \sigma^2)$ with $\lambda = (\mu, \sigma)$ and

$$z = g(\lambda, \epsilon) = \mu + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0,1).$$

ELBO integrand:

$$f(z) = \log p(x,z) - \log q_\lambda(z) = -\tfrac{1}{2}(x-z)^2 - \tfrac{1}{2}z^2 \; - \; \Big(-\tfrac{1}{2}\log(2\pi\sigma^2) - \tfrac{1}{2}\frac{(z-\mu)^2}{\sigma^2} \Big) + C.$$

Pathwise gradients:

$$\nabla_z \log p(x,z) = (x-z) - z = x - 2z, \quad \nabla_z \log q_\lambda(z) = -\frac{z-\mu}{\sigma^2}.$$

$$\nabla_\mu g = 1, \qquad \nabla_\sigma g = \epsilon.$$

SG estimator (single sample):

$$\widehat{\nabla_\mu \mathcal{L}} = \big[x - 2z + \tfrac{z-\mu}{\sigma^2}\big]\Big|_{z=\mu+\sigma\epsilon}, \quad \widehat{\nabla_\sigma \mathcal{L}} = \big[x - 2z + \tfrac{z-\mu}{\sigma^2}\big]\Big|_{z=\mu+\sigma\epsilon} \cdot \epsilon.$$

Update: $\mu \leftarrow \mu + \eta\, \widehat{\nabla_\mu \mathcal{L}}, \; \sigma \leftarrow \sigma + \eta\, \widehat{\nabla_\sigma \mathcal{L}}.$

Two ways to automate VI: score-function vs. pathwise gradients

1. Score-function (REINFORCE, black-box VI)

$$\nabla_\lambda \mathcal{L} = \mathbb{E}_{q_\lambda}\big[\nabla_\lambda \log q_\lambda(z)\,\big(\log p(x,z) - \log q_\lambda(z)\big)\big]$$

- Differentiates the **density** q_λ; the model is only *evaluated*.
- Works for **discrete and continuous** z, non-differentiable models.
- **Downside:** high variance; needs control variates, larger S.

2. Pathwise (reparameterization)

$$\nabla_\lambda \mathcal{L} = \mathbb{E}_\epsilon\big[\nabla_\lambda\big(\log p(x, g(\lambda,\epsilon)) - \log q_\lambda(g(\lambda,\epsilon))\big)\big]$$

- Differentiates the **sample path** $z = g(\lambda,\epsilon)$ by autodiff.
- Needs **continuous, reparameterizable** q_λ and differentiable $\log p(x,z)$.
- **Upside:** typically much lower variance; the default in PyTorch, Pyro/NumPyro, Stan ADVI.

Common to both

Same objective (ELBO), same loop (sample $\to$ estimate $\widehat{\nabla_\lambda \mathcal{L}}$ $\to$ SGD/Adam), minibatches with N/k scaling, and **any family q_λ they can handle, including mean-field**.

How do we choose q_λ for automated VI?

A practical order (start simple, add structure if needed)

1. Constrained unknowns ($\sigma > 0$, $p \in (0,1)$): fit the Gaussian to $u = \log \sigma$ or $u = \text{logit}\, p$, which can be any real number, and map samples back ($\sigma = e^u$, $p = \text{sigmoid}(u)$). Add $\log \left| \frac{d\sigma}{du} \right|$ ($= u$ for the log) to $\log p$ so the prior is unchanged.
2. **Mean-field Gaussian** first: $2d$ parameters, cheap, robust.
3. **Correlations matter** (e.g. slope/intercept, hierarchical scales) and d is up to a few thousand: **full-rank** Gaussian; larger d: **low-rank + diagonal**.
4. **Skewed or multimodal** posterior: mixtures or normalizing flows.
5. **Discrete latents:** sum them out if possible; otherwise score-function gradients.

Which gradient estimator?

- Continuous z, differentiable model: **pathwise**.
- Discrete z or a black-box simulator: **score function**.

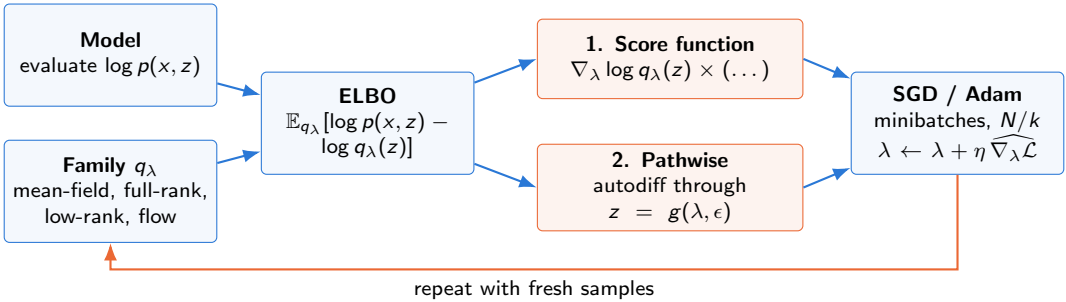
Is q good enough?

- Repeat runs from different initializations.
- Compare a richer family: does the answer change?
- Posterior predictive checks; PSIS $\hat{k}$ diagnostic (Yao et al., 2018).
- Small problem: compare with MCMC (NUTS).

Limitations of variational inference

- **Approximation bias.** VI returns the best member of the family, not the posterior. More iterations or samples do not remove the gap (unlike MCMC, which is exact in the limit).
- **Reverse KL is mode-seeking.** It underestimates variance and can drop modes; mean-field also drops correlations $\Rightarrow$ overconfident intervals.
- **Non-convex optimization.** Local optima, sensitivity to initialization, learning rate, S and k; a loss curve that has flattened does not prove a good q.
- **No absolute accuracy measure.** $\log p(x)$ is unknown, so the ELBO only compares candidates. Use diagnostics: PSIS $\hat{k}$, predictive checks, comparison with MCMC.
- **Estimator requirements.** Pathwise needs continuous, reparameterizable q and a differentiable model; score function works more generally but is noisy.
- **Difficult geometry.** Hierarchical models (funnels) and heavy tails are hard for Gaussian families.

Take-home: VI can be automated in two ways



- **Automatic:** once we can evaluate $\log p(x, z)$ and sample from q_λ, nothing model-specific needs deriving.
- The two approaches differ only in **how the gradient is estimated**. The family (e.g. **mean-field**) is a **separate** choice that works with both.
- CAVI is the hand-derived special case for conjugate, mean-field models.

References for Variational Inference and Reparameterization

Core textbook

- D. MacKay (2003), *Information Theory, Inference, and Learning Algorithms*, Ch. 33 "Variational Methods", https://www.inference.org.uk/itprnn/book.pdf

Key review

- D. Blei, A. Kucukelbir, J. McAuliffe (2017), "Variational Inference: A Review for Statisticians", *JASA*, https://arxiv.org/abs/1601.00670

Advanced tutorial

- T. Broderick (2018), "Variational Bayes and Beyond: Bayesian Inference for Big Data", ICML Tutorial, https://tamarabroderick.com/tutorial_2018_icml.html

Online course

- P. Huijse et al., *Bayesian Learning and Neural Networks*, https://phuijse.github.io/BLNNbook/README.html

Course slide reference

- M. Villani (2024), *Advanced Bayesian Learning, Lecture 6: Beyond Mean-Field VI*.

Automated VI and gradient estimators

- R. Ranganath, S. Gerrish, D. Blei (2014), "Black Box Variational Inference", *AISTATS*.
- A. Kucukelbir, D. Tran, R. Ranganath, A. Gelman, D. Blei (2017), "Automatic Differentiation Variational Inference", *JMLR*.
- M. Hoffman, D. Blei, C. Wang, J. Paisley (2013), "Stochastic Variational Inference", *JMLR*.
- Y. Yao, A. Vehtari, D. Simpson, A. Gelman (2018), "Yes, but Did It Work?: Evaluating Variational Inference", *ICML*.
- Y. Shen et al. (2024), "Variational Learning is Effective for Large Deep Networks", *ICML*.

Acknowledgements/ Document preparation

This document was developed and converted into LaTeX slides and formatted with assistance from ChatGPT (OpenAI) and CoPilot (Microsoft). The instructor modified the source text and verified the final structure and wording.

Appendix

Optional and background slides

Algorithm: mini-batch sampling with replacement

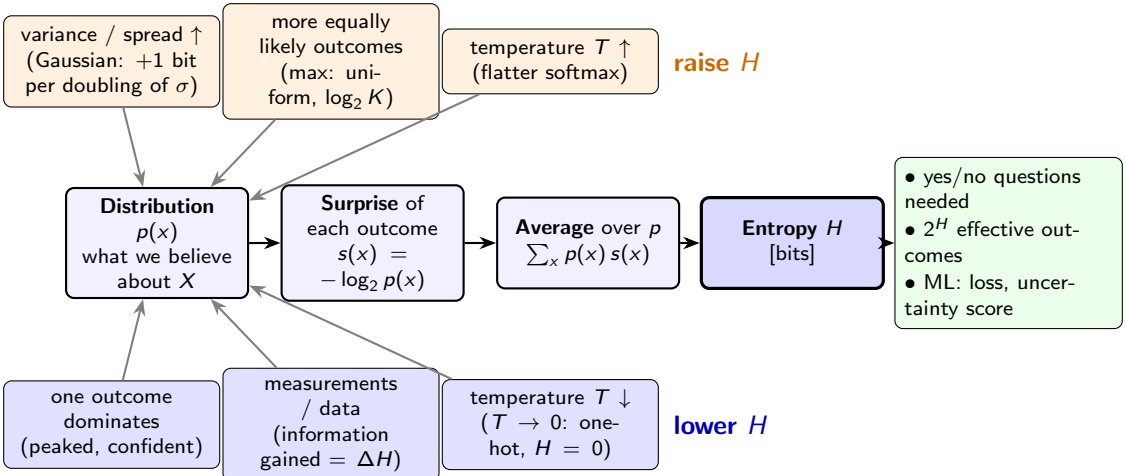
Algorithm 2 Mini-batch SGD with replacement (clean unbiased estimator)

Require: Dataset size n, batch size k, steps T, learning rates $\{\eta_t\}_{t=0}^{T-1}$

1: Initialize parameters θ
2: **for** $t = 0$ to $T - 1$ **do**
3: Sample indices $i_{t,1}, \ldots, i_{t,k} \overset{\text{i.i.d.}}{\sim} \text{Unif}(\{1, \ldots, n\})$
4: $g \leftarrow \frac{1}{k} \sum_{j=1}^{k} \nabla_\theta \ell(\theta; x_{i_{t,j}}, y_{i_{t,j}})$
5: $\theta \leftarrow \theta - \eta_t \, g$
6: **end for**

- **No epochs**: each step is independent. Same compute as E epochs: $T = E \lceil n/k \rceil$.
- In n draws only $\approx 1 - 1/e \approx 63\%$ of the examples are seen; some twice, $\approx 37\%$ not at all.
- Used in convergence proofs, weighted / imbalanced sampling, and very large or streaming data (e.g. LLM pretraining, often < 1 epoch, measured in steps).
- Set $k = 1$ in either algorithm to obtain "single-sample" SGD.

How entropy is computed, and what moves it



H is a property of the *distribution*, not of one outcome: it measures our uncertainty *before* we observe X.

Cross-Entropy and NLL in Classification (Derivation)

Setup.

- True data distribution: $p_{\text{data}}(x, y)$.
- Model: conditional probabilities $p_\theta(y \mid x)$.
- Per-example negative log-likelihood (NLL):

$$\text{NLL}(x, y) = -\log p_\theta(y \mid x).$$

Expected NLL under the true distribution:

$$\mathbb{E}_{(X,Y)\sim p_{\text{data}}}[\text{NLL}(X, Y)] = \mathbb{E}_{p_{\text{data}}(x,y)}[-\log p_\theta(y \mid x)].$$

We can write this as a two-step expectation:

$$\begin{aligned}\mathbb{E}_{p_{\text{data}}(x,y)}[-\log p_\theta(y \mid x)] &= \mathbb{E}_{p_{\text{data}}(x)}\Big[\mathbb{E}_{p_{\text{data}}(y|x)}[-\log p_\theta(y \mid x)]\Big] \\ &= \mathbb{E}_{p_{\text{data}}(x)}\Big[H\big(p_{\text{data}}(\cdot \mid x), p_\theta(\cdot \mid x)\big)\Big],\end{aligned}$$

where for each fixed x we define the *cross-entropy*

$$H\big(p_{\text{data}}(\cdot \mid x), p_\theta(\cdot \mid x)\big) = -\sum_y p_{\text{data}}(y \mid x)\, \log p_\theta(y \mid x).$$

Cross-Entropy, Entropy, KL and NLL (General Case)

For each fixed x, we have the identity

$$H\big(p_{\text{data}}(\cdot \mid x), p_\theta(\cdot \mid x)\big) = H\big(p_{\text{data}}(\cdot \mid x)\big) + \mathrm{KL}\big(p_{\text{data}}(\cdot \mid x) \,\|\, p_\theta(\cdot \mid x)\big),$$

where

$$H\big(p_{\text{data}}(\cdot \mid x)\big) = -\sum_y p_{\text{data}}(y \mid x)\, \log p_{\text{data}}(y \mid x)$$

is the (conditional) entropy of the true labels given x.

Taking expectation over x:

$$\mathbb{E}_{p_{\text{data}}(x,y)}[-\log p_\theta(y \mid x)] = H_{p_{\text{data}}}(Y \mid X) + \mathbb{E}_{p_{\text{data}}(x)}\Big[\mathrm{KL}\big(p_{\text{data}}(\cdot \mid x) \,\|\, p_\theta(\cdot \mid x)\big)\Big],$$

where

$$H_{p_{\text{data}}}(Y \mid X) = \mathbb{E}_{p_{\text{data}}(x)}\Big[H\big(p_{\text{data}}(\cdot \mid x)\big)\Big].$$

Interpretation and generality

- $H_{p_{\text{data}}}(Y \mid X)$: intrinsic label uncertainty (Bayes error).
- The KL term: how far the model $p_\theta(y \mid x)$ is from the true conditional $p_{\text{data}}(y \mid x)$.
- This derivation is valid for *any discrete label space*: binary, multi-class, or any finite set.

LogNormal example: Jensen quantities as functions of the parameter s

Distribution. Let $X \sim \mathrm{LogNormal}(m, s^2)$, i.e.

$$\log X \sim \mathcal{N}(m, s^2), \qquad X > 0.$$

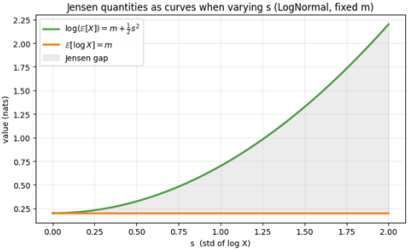
Closed-form expectations (fix m).

- $\mathbb{E}[\log X] = m$ *(flat line in s)*
- $\mathbb{E}[X] = \exp\left(m + \frac{1}{2}s^2\right)$
- $\log(\mathbb{E}[X]) = m + \frac{1}{2}s^2$ *(curves upward in s)*

Jensen gap.

$$\log(\mathbb{E}[X]) - \mathbb{E}[\log X] = \tfrac{1}{2}s^2 \;\geq\; 0,$$

with equality only when $s = 0$ (degenerate case).



Jensen geometry for log: points at $x = \mathbb{E}[X]$ (as x varies)

For any positive random variable X (with finite expectations),

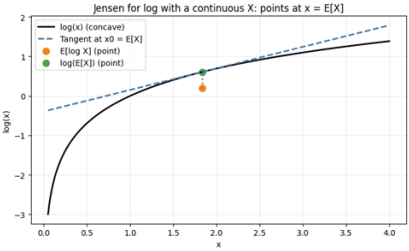
$$\log(\mathbb{E}[X]) \;\geq\; \mathbb{E}[\log X].$$

Geometric proof idea (concavity). Because log is concave, the tangent line at $x_0 = \mathbb{E}[X]$ lies above the curve:

$$\log(x) \;\leq\; \log(x_0) + \frac{1}{x_0}(x - x_0), \qquad x_0 = \mathbb{E}[X].$$

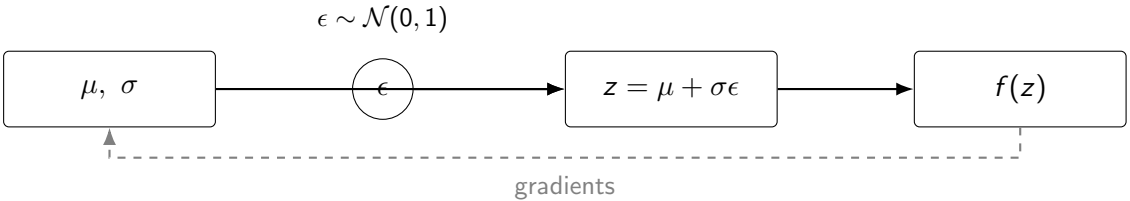
Taking expectations of both sides gives Jensen's inequality. **How to read the plot.**

- Both quantities are **single numbers** once the distribution is fixed.
- They are shown as two **points** at the same $x = \mathbb{E}[X]$: $(\mathbb{E}[X],\, \mathbb{E}[\log X])$ and $(\mathbb{E}[X],\, \log(\mathbb{E}[X]))$.
- The vertical gap is $\log(\mathbb{E}[X]) - \mathbb{E}[\log X] \geq 0$.



- E[X] = 1.83
- E[log X] = 0.2
- log(E[X]) = 0.61
- gap = 0.41

Reparameterization computation graph



Backprop flows through $z = \mu + \sigma\epsilon$; randomness is isolated in ϵ.

Intuition. Move randomness to ϵ and make z a deterministic function of (μ, σ, ϵ).
Why it helps. Backprop can compute $\partial f/\partial\mu$ and $\partial f/\partial\sigma$ through z.
Result. Typically much lower-variance gradients than REINFORCE.

Control variates / baselines

Subtract a baseline b (independent of z):

$$\mathbb{E}[(f(z) - b)\nabla_\lambda \log q_\lambda(z)] = \mathbb{E}[f(z)\nabla_\lambda \log q_\lambda(z)].$$

So the estimator stays unbiased but can have much lower variance.

- Common choice: $b \approx \mathbb{E}[f(z)]$ (running mean)
- More advanced: learned baseline / critic (RL)